

Паттерны программирования

Лекция 2

Примеры объектно-
ориентированного подхода в ruby

Учебные вопросы

1. Функции и способы их задания.
2. Понятие полиморфизма. Модули в Ruby
3. Встроенные модули языка. Структуры данных в Ruby
4. Исключения в ruby

Понятие блока

Выделенные строки
повторяются в трех
методах.

Впрочем, строка
в середине отлича-
ется...

Отличается...

Отличается...

```
def total(prices)
  amount = 0
  index = 0
  while index < prices.length
    amount += prices[index]
    index += 1
  end
  amount
end
```

```
def refund(prices)
  amount = 0
  index = 0
  while index < prices.length
    amount -= prices[index]
    index += 1
  end
  amount
end
```

```
def show_discounts(prices)
  index = 0
  while index < prices.length
    { amount_off = prices[index] / 3.0
      puts format("Your discount: $%.2f", amount_off)
      index += 1
    }
  end
end
```

Функции и методы

В параметре этого метода передается блок!

```
def my_method(&my_block)
  puts "We're in the method, about to invoke your block!"
  my_block.call ← Метод «call» вызывает переданный блок.
  puts "We're back in the method!"
end
```

Функции и методы

Блок всегда должен
следовать за вызовом
метода.

Начало блока.

Тело блока.

```
my_method
```

```
do
```

```
  puts "We're in the block!"
```

```
end
```

Конец блока.

Функции и методы

Объявление
другого мето-
да, получаю-
щего блок.

```
def twice(&my_block)
  puts "In the method, about to call the block!"
  my_block.call ← Вызывается блок.
  puts "Back in the method, about to call the block again!"
  my_block.call ← СНОВА вызывается блок.
  puts "Back in the method, about to return!"
end
```

Вызов метода
с передачей блока.

```
twice do
  puts "Wooooo!"
end
```

Как видно из результата, метод действительно вызывает блок дважды!

```
In the method, about to call the block!
Wooooo!
Back in the method, about to call the block again!
Wooooo!
Back in the method, about to return!
```

Функции и методы

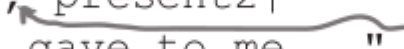
```
def give(&my_block)
  my_block.call("2 turtle doves", "1 partridge")
end
```



Параметр 1

Параметр 2

```
give do |present1, present2|
  puts "My method gave to me..."
  puts present1, present2
end
```



Если параметров
несколько, они раз-
деляются запятыми.

Функции и методы

```
def twice(&my_block)
  my_block.call
  my_block.call
end
```

```
def twice
  yield
  yield
end
```

```
def give(&my_block)
  my_block.call("2 turtle doves", "1 partridge")
end
```

```
def give
  yield "2 turtle doves", "1 partridge"
end
```


Функции и методы

```
def take_this  
  yield "present"  
end
```

```
take_this do |thing|  
  puts "do/end block got #{thing}"  
end
```

```
take_this { |thing| puts "braces block got #{thing}" }
```

Понятие блока

Выделенные строки
повторяются в трех
методах.

Впрочем, строка
в середине отлича-
ется...

Отличается...

Отличается...

```
def total(prices)
  amount = 0
  index = 0
  while index < prices.length
    amount += prices[index]
    index += 1
  end
  amount
end
```

```
def refund(prices)
  amount = 0
  index = 0
  while index < prices.length
    amount -= prices[index]
    index += 1
  end
  amount
end
```

```
def show_discounts(prices)
  index = 0
  while index < prices.length
    { amount_off = prices[index] / 3.0
      puts format("Your discount: $%.2f", amount_off)
    }
    index += 1
  end
end
```

Функции и методы

```
class Array
```

```
  def each
```

```
    {index = 0
```

```
    while index < self.length
```

```
      yield self[index]
```

```
      index += 1
```

```
    end
```

```
  end
```

```
end
```

Почти не отличается от циклов в наших методах «total», «refund» и «show_discounts»!

Вспомните: «self» обозначает текущий объект — в данном случае текущий массив.

Главное отличие: текущий элемент передается блоку!

Затем происходит переход к следующему элементу, как и прежде.

Функции и методы

```
def total(prices)
  amount = 0
  prices.each do |price|
    amount += price
  end
  amount
end

def refund(prices)
  amount = 0
  prices.each do |price|
    amount -= price
  end
  amount
end

def show_discounts(prices)
  prices.each do |price|
    amount_off = price / 3.0
    puts format("Your discount: $%.2f", amount_off)
  end
end

prices = [3.99, 25.00, 8.99]

puts format("%.2f", total(prices))
puts format("%.2f", refund(prices))
show_discounts(prices)
```

Начинаем с 0.

Обработываем каждую цену.

Текущая цена прибавляется к сумме.

Возвращаем итоговое значение.

Начинаем с 0.

Обработываем каждую цену.

Вычитаем текущую цену.

Возвращаем итоговое значение.

Обработываем каждую цену.

Вычисляем скидку.

Форматируем и выводим текущую скидку.



Функции и методы

```
def triple_block_result
  puts 3 * yield
end
```

Блок возвращает 2.

```
triple_block_result { 2 }
triple_block_result { 5 }
```

Блок возвращает 5.



Или использовать его в строке:

```
def greet
  puts "Hello, #{yield}!"
end
```

Блок возвращает эту строку.

```
greet { "Liz" }
```



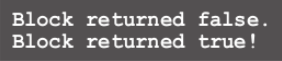
Или использовать его в условной команде:

```
def alert_if_true
  if yield
    puts "Block returned true!"
  else
    puts "Block returned false."
  end
end
```

Блок возвращает false.

```
alert_if_true { 2 + 2 == 5 }
alert_if_true { 2 > 1 }
```

Блок возвращает true.



Функции и методы

Если бы мы захотели написать собственную версию `find_all`, она могла бы выглядеть примерно так:

```
def find_all
  matching_items = []
  self.each do |item|
    if yield(item)
      matching_items << item
    end
  end
  matching_items
end
```

Создание нового массива для хранения элементов, для которых блок возвращает true.

Обработка всех элементов.

Элемент передается блоку.
Если блок возвращает true...

...то элемент добавляется в массив подходящих элементов.

```
relevant_lines = lines.find_all { |line| line.include?("String") }
```

Функции и методы

Если бы мы решили написать собственную реализацию `reject`, она получилась бы очень похожей на `find_all`:

```
def reject
  kept_items = []
  self.each do |item|
    unless yield(item)
      kept_items << item
    end
  end
  kept_items
end
```

Создание нового массива для элементов, для которых блок возвращает `false`.

Обработка каждого элемента.

Элемент передается блоку. Если возвращаемый результат равен `false`...

...элемент добавляется в массив сохраняемых элементов.

```
reviews = relevant_lines.reject { |line| line.include?("--")
}
```

Функции и методы

Если бы мы решили написать собственную реализацию `reject`, она получилась бы очень похожей на `find_all`:

```
def reject
  kept_items = []
  self.each do |item|
    unless yield(item)
      kept_items << item
    end
  end
  kept_items
end
```

Создание нового массива для элементов, для которых блок возвращает `false`.

Обработка каждого элемента.

Элемент передается блоку. Если возвращаемый результат равен `false`...

...элемент добавляется в массив сохраняемых элементов.

```
reviews = relevant_lines.reject { |line| line.include?("--")
}
```


Функции и методы

Создавать массивы результатов заранее не нужно — «map» создаст их за нас!

```
squares = [2, 3, 4].map { |number| number ** 2 }
cubes = [2, 3, 4].map { |number| number ** 3 }
area_codes = ['1-800-555-0199', '1-402-555-0123'].map do |phone|
  phone.split("-")[1]
end
p squares, cubes, area_codes
```

Создание нового массива с квадратами чисел.

Создание нового массива с кубами чисел.

Создание нового массива с кодами зон.

```
[4, 9, 16]
[8, 27, 64]
["800", "402"]
```

Функции и методы

```
class Candidate
  attr_accessor :name, :age, :occupation, :hobby, :birthplace
  def initialize(name, age, occupation, hobby, birthplace)
    self.name = name
    self.age = age
    self.occupation = occupation
    self.hobby = hobby
    self.birthplace = birthplace
  end
end
```

Пара-метры используются для назначения атрибутов объекта.

← к атрибутам.

← Настройка передачи аргументов Candidate.new.

Добавим после определения класса код создания экземпляра Candidate и вывода его данных.

```
def print_summary(candidate)
  puts "Candidate: #{candidate.name}"
  puts "Age: #{candidate.age}"
  puts "Occupation: #{candidate.occupation}"
  puts "Hobby: #{candidate.hobby}"
  puts "Birthplace: #{candidate.birthplace}"
end
```

```
candidate = Candidate.new("Carl Barnes", 49, "Attorney", nil, "Miami")
print_summary(candidate)
```

Аргумент должен передаваться даже в том случае, если он не используется.

Функции и методы

```
class Candidate
  attr_accessor :name, :age, :occupation, :hobby, :birthplace
  def initialize(name, age, occupation, hobby, birthplace)
    self.name = name
    self.age = age
    self.occupation = occupation
    self.hobby = hobby
    self.birthplace = birthplace
  end
end
```

Пара-метры используются для назначения атрибутов объекта.

← к атрибутам.

← Настройка передачи аргументов Candidate.new.

Добавим после определения класса код создания экземпляра Candidate и вывода его данных.

```
def print_summary(candidate)
  puts "Candidate: #{candidate.name}"
  puts "Age: #{candidate.age}"
  puts "Occupation: #{candidate.occupation}"
  puts "Hobby: #{candidate.hobby}"
  puts "Birthplace: #{candidate.birthplace}"
end
```

```
candidate = Candidate.new("Carl Barnes", 49, "Attorney", nil, "Miami")
print_summary(candidate)
```

Аргумент должен передаваться даже в том случае, если он не используется.

Функции и методы

```
class Candidate
  attr_accessor :name, :age, :occupation, :hobby, :birthplace
  def initialize(name, age, occupation, hobby = nil, birthplace)
    ...
  end
end
```

↑ Значение по умолчанию,
чтобы сделать параметр
необязательным...

АРГУМЕНТЫ ПО УМОЛЧАНИЮ ВСЕГДА В КОНЦЕ

Функции и методы

Ниже приведена переработанная версия метода initialize класса Candidate с параметром-хешем.

```
class Candidate
  attr_accessor :name, :age, :occupation, :hobby, :birthplace
  def initialize(name, options)
    self.name = name
    self.age = options[:age]
    self.occupation = options[:occupation]
    self.hobby = options[:hobby]
    self.birthplace = options[:birthplace]
  end
end
```

Имя хранится в отдельной строке.

Имя присваивается, как обычно.

Получаем значения из хеша, а не напрямую из параметров.

Параметр-хеш

Теперь при вызове Candidate.new сначала передается строка с именем, а затем следует хеш со значениями всех остальных атрибутов Candidate:

```
candidate = Candidate.new("Amy Nguyen",
  { :age => 37, :occupation => "Engineer", :hobby => "Lacrosse", :birthplace => "Seattle" })
```

Теперь понятно, какой атрибут соответствует каждому аргументу!

p candidate

```
#<Candidate:0x007fbd7a02e858 @name="Amy Nguyen", @age=37,
@occupation="Engineer", @hobby="Lacrosse", @birthplace="Seattle">
```

Теперь ошибки с порядком исключены!

Ключи хеша при желании можно опустить. В этом случае атрибуту будет присвоен объект хеша по умолчанию, nil.

```
candidate = Candidate.new("Carl Barnes",
  { :age => 49, :occupation => "Attorney", :birthplace => "Miami" })
```

Атрибут hobby можно не указывать.

p candidate

```
#<Candidate:0x007f8aaa042a68 @name="Carl Barnes", @age=49,
@occupation="Attorney", @hobby=nil, @birthplace="Miami">
```

По умолчанию пропущенным атрибутам присваивается nil.

Наконец, ключи хеша могут следовать в произвольном порядке:

```
candidate = Candidate.new("Amy Nguyen",
  { :birthplace => "Seattle", :hobby => "Lacrosse", :occupation => "Engineer", :age => 37 })
```

Функции и методы

```
candidate = Candidate.new("Carl Barnes",  
  :age => 49, :occupation => "Attorney")  
p candidate
```

Без фигурных скобок!

```
#<Candidate:0x007fb412802c30  
  @name="Carl Barnes", @age=49,  
  @occupation="Attorney",  
  @hobby=nil, @birthplace=nil>
```

```
candidate = Candidate.new("Amy Nguyen", age: 37,  
  occupation: "Engineer", hobby: "Lacrosse")  
p candidate
```

*Те же символические имена,
но читаются гораздо лучше!*

```
#<Candidate:0x007f9dc412aa98  
  @name="Amy Nguyen", @age=37,  
  @occupation="Engineer",  
  @hobby="Lacrosse",  
  @birthplace=nil>
```

Функции и методы

```
class Candidate
  attr_accessor :name, :age, :occupation, :hobby, :birthplace
  def initialize(name, options = {}) ← Если хеш не передается,
    self.name = name                  использовать пустой хеш.
    self.age = options[:age]
    self.occupation = options[:occupation]
    self.hobby = options[:hobby]
    self.birthplace = options[:birthplace]
  end
end
```

Если аргумент-хеш не передается, по умолчанию будет использоваться пустой хеш. Всем атрибутам Candidate будет присвоено значение по умолчанию nil из пустого хеша.

```
p Candidate.new("Carl Barnes")
```

```
#<Candidate:0x007fbe0981ec18 @name="Carl Barnes", @age=nil,
@occupation=nil, @hobby=nil, @birthplace=nil>
```

Функции и методы

```
class Candidate
  attr_accessor :name, :age, :occupation, :hobby, :birthplace
  def initialize(name, age: nil, occupation: nil, hobby: nil, birthplace: "Sleepy Creek")
    self.name = name
    self.age = age
    self.occupation = occupation
    self.hobby = hobby
    self.birthplace = birthplace
  end
end
```

Параметр-хеш заменяется
ключевыми словами и зна-
чениями по умолчанию.

Вместо ключей хеша исполь-
зуются имена параметров.

Функции и методы

```
class Candidate
  attr_accessor :name, :age, :occupation, :hobby, :birthplace
  def initialize(name, age:, occupation:, hobby: nil, birthplace: "Sleepy Creek")
    self.name = name
    self.age = age
    self.occupation = occupation
    self.hobby = hobby
    self.birthplace = birthplace
  end
end
```

Ключевые слова без значений по умолчанию!



Функции и методы

```
def two_plus( one, two, *args )
  length = args.size
  label = length == 1 ? " variable argument" : " variable arguments"
  num = length.to_s + label + "(" + args.inspect + ")"
  num
end

puts two_plus( 1, 2 )
puts two_plus( 1000, 3.5, 14.3 )
puts two_plus( 100, 2.5, "three", 70, 14.3 )
```

Вот результаты (показано только, сколько получено переменных параметров; обычные параметры игнорируются):

```
0 variable arguments ([])
1 variable argument ([14.3])
3 variable arguments (["three", 70, 14.3])
```

Функции и методы

```
irb(main):001:0> def greet
irb(main):002:1>   puts "Hello, baby!"
irb(main):003:1> end
=> nil

irb(main):004:0> alias baby greet # для greet альтернативное имя baby
=> nil

irb(main):005:0> greet    # вызов метода
Hello, baby!
=> nil

irb(main):006:0> baby    # вызов по альтернативному имени
Hello, baby!
=> nil

irb(main):007:0> greet.object_id    # каков идентификатор объекта?
Hello, baby!
=> 4

irb(main):008:0> baby.object_id # указывает на тот же самый объект
Hello, baby!
=> 4
```

Ruby: Что такое lambda-функции

Лямбда (lambda) функции — грубо говоря, методы без названия. Вместо объявления метода вы как бы записываете его в переменную.

При записывании функции в переменную, нужно передать блок (в том числе с параметрами), который будет выполнен при вызове этой функции:

```
function = lambda { |x| x ** 2 }
```

Чтобы вызвать lambda-функцию, необходимо у объекта (переменной, в которой записана lambda-функция) вызвать метод **call** с нужным набором аргументов:

```
> function.call(16)  
=> 256
```

Функции и методы

```
count = Proc.new { [1,2,3,4,5].each do |i| print i end; puts }  
your_proc = lambda { puts "Lurch: 'You rang?'" }  
my_proc = proc { puts "Morticia: 'Who was at the door, Lurch?'" }  
  
# Какой тип объектов вы только что создали?  
puts count.class, your_proc.class, my_proc.class  
  
# Вызов всех "проков"  
count.call  
your_proc.call  
my_proc.call  
  
12345  
Lurch: 'You rang?'  
Morticia: 'Who was at the door, Lurch?'
```

Полиморфизм

```
module MyMod

  def meth1
    puts "Это метод 1"
  end

end

class MyClass

  include MyMod
  # ...
end

x = MyClass.new
x.meth1                # Это метод 1
```

Полиморфизм

```
module MyMod

  def self.included(klass)
    def klass.module_method

      puts "Module (class) method"
    end
  end

  def method_1
    puts "Method 1"
  end

end

class MyClass

  include MyMod

  def self.class_method
    puts "Class method"
  end

  def method_2
    puts "Method 2"
  end

end

x = MyClass.new

# Выводится:
MyClass.class_method # Class method
x.method_1           # Method 1
MyClass.module_method # Module (class) method
x.method_2           # Method 2
```

```
Address = Struct.new("Address", :street, :city, :state)
books = Address.new("411 Elm St", "Dallas", "TX")

Struct.new("Address", :street, :city, :state)
books = Struct::Address.new("411 Elm St", "Dallas", "TX")
```

```
class Address

  attr_accessor :street, :city, :state

  def initialize(street1, city, state)
    @street, @city, @state = street, city, state
  end

end

books = Address.new("411 Elm St", "Dallas", "TX")
```

```
class Person
  attr_reader :name, :age, :height

  def initialize(name, age, height)
    @name, @age, @height = name, age, height
  end

  def inspect
    "#@name #@age #@height"
  end

end

def map_by(sym)
  self.map {|x| x.send(sym) }
end

people = []
people << Person.new("Hansel", 35, 69)
people << Person.new("Gretel", 32, 64)
people << Person.new("Ted", 36, 68)
people << Person.new("Alice", 33, 63)

p1 = people.map_by(:name) # Hansel, Gretel, Ted, Alice
p2 = people.map_by(:age)  # 35, 32, 36, 33
p3 = people.map_by(:height) # 69, 64, 68, 63
```

Модули

```
class MySuperclass
end

module MyModule
  def my_method
    puts "hello from MyModule"
  end
end

class MyClass < MySuperclass
  include MyModule
end
```

```
p MyClass.ancestors
```

Примесь

```
[MyClass, MyModule, MySuperclass, Object, Kernel, BasicObject]
```

```
module MyModule
  def my_method
    puts "hello from MyModule"
  end
end
```

```
class MyClass
  include MyModule
  def my_method
    puts "hello from MyClass"
  end
end
```

```
p MyClass.ancestors
```

Переопределенный метод
находится здесь.

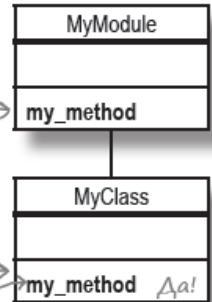
И до этого места
вообще не доходит!

```
[MyClass, MyModule, Object, Kernel, BasicObject]
hello from MyClass
```

Никогда
не вызы-
вается!

Здесь
поиск
прекра-
щается.

Есть метод «my_method»? Да!



Модули

```
module AcceptsComments
  attr_reader :comments
  def initialize
    @comments = []
  end
  def add_comment(comment)
    comments << comment
  end
end

class Photo
  include AcceptsComments
  def initialize
    @format = 'JPEG'
  end
end

photo = Photo.new
photo.add_comment("Beautiful colors.")
```

Переменная экземпляра `@comments` содержит `nil`, поэтому здесь происходит ошибка.

Переопределяет метод «initialize» в примеси, поэтому тот не будет выполнен.

Ошибка → `in 'add comment': undefined method '<<' for nil:NilClass`

Присваиваем новое значение `@comments`, но только в том случае, если `@comments` содержит `nil`.

```
module AcceptsComments
  def comments
    if @comments
      @comments
    else
      @comments = []
    end
  end
end
```

Присваиваем новое значение `@comments`, но только в том случае, если `@comments` содержит `nil`.

```
module AcceptsComments
  def comments
    @comments = @comments || []
  end
end
```

Та же функциональность в одной строке!

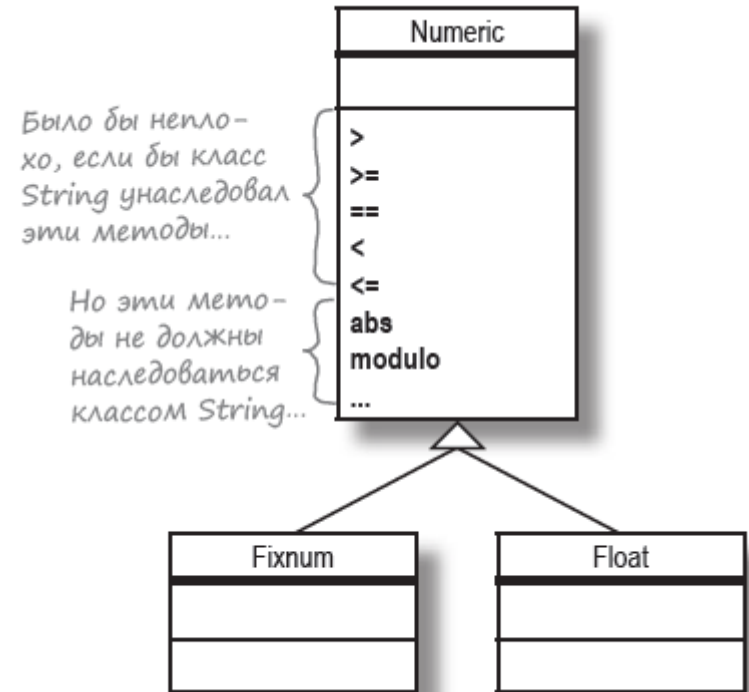
Модули

```
module AcceptsComments
  def comments
    @comments ||= [] ← Если переменная экземпляра @comments
  end                                     содержит nil, ей присваивается пу-
  def add_comment(comment)              стой массив. Метод возвращает значе-
    comments << comment                 ние @comments.
  end
end

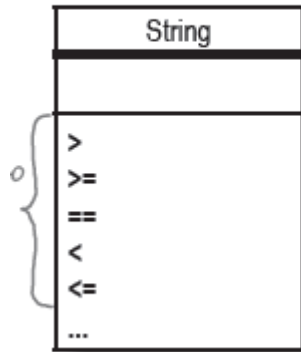
class Photo
  include AcceptsComments
  def initialize
    @format = 'JPEG'
  end
end

photo = Photo.new
photo.add_comment("Beautiful colors.")
p photo.comments
```

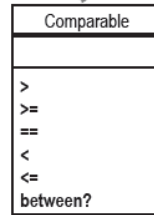
["Beautiful colors."]



Модули

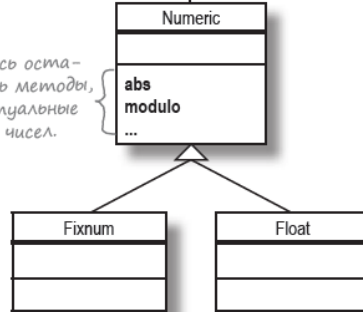


Включается в класс Numeric



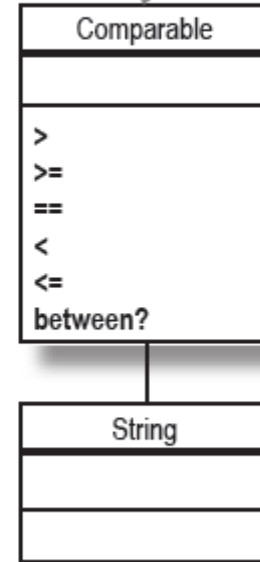
Помните: это не наследовани поэтому тре! ника здесь не п

Здесь оста-
лись методы,
актуальные
для чисел.



Эти классы на-
следуют все ме-
тоды из примесей
Comparable и Numeric.

Включается
в класс String.



Модули

```
module Comparable
  def <(other)
    (self <=> other) == -1
  end
  def >(other)
    (self <=> other) == 1
  end
  def ==(other)
    (self <=> other) == 0
  end
  def <=(other)
    comparison = (self <=> other)
    comparison == -1 || comparison == 0
  end
  def >=(other)
    comparison = (self <=> other)
    comparison == 1 || comparison == 0
  end
  def between?(first, second)
    (self <=> first) >= 0 && (self <=> second) <= 0
  end
end
```

Конечно, «self» обозначает текущий экземпляр.

Как и прежде, «other» — другой экземпляр, с которым сравнивается экземпляр this.

Все остальные методы используют «other» и «self» аналогичным образом.

Модули МЕТОД

```
def <=>(other)
```

```
//
```

```
end
```

обязательно реализовать

Модули

Методы экземпляра из Enumerable:

all?	find_all	none?
any?	find_index	one?
chunk	first	partition
collect	flat_map	reduce
collect_concat	grep	reject
count	group_by	reverse_each
cycle	include?	select
detect	inject	slice_before
drop	lazy	sort
drop_while	map	sort_by
each_cons	max	take
each_entry	max_by	take_while
each_slice	member?	to_a
each_with_index	min	to_h
each_with_object	min_by	to_set
entries	minmax	zip
find	minmax_by	

Модули

Comparable:

- Предоставляет `<`, `>`, `==` и еще три метода.
- Включается классами `String`, `Fixnum` и другими числовыми классами.
- Включающий класс должен предоставить метод `<=>`.

Enumerable:

- Предоставляет методы `find_all`, `reject`, `map` и еще 47 других методов.
- Включается классами `Array`, `Hash` и другими классами коллекций.
- Включающий класс должен предоставить метод `each`.

Модули

```
class WordSplitter
```

```
  attr_accessor :string
```

Для хранения строки,
разделяемой на слова.

```
  def each
```

Будет вызываться методами Enumerable.

```
    string.split(" ").each do |word|
```

```
      yield word
```

```
    end
```

```
  end
```

```
end
```

Разделяем строку на слова (по пробелам) и обрабатываем каждое слово.

Текущее слово передается блоку, который был передан «each».

```
splitter = WordSplitter.new
```

```
splitter.string = "one two three four"
```

Строка, разделяемая на слова.

```
splitter.each do |word|
```

```
  puts word
```

```
end
```

Каждое слово передается блоку в параметре.

Выводим текущее слово.

```
one
two
three
four
```


Модули

```
class WordSplitter
```

```
  include Enumerable
```

← Включаем Enumerable.

```
  attr_accessor :string
```

```
  def each
    string.split(" ").each do |word|
      yield word
    end
  end
end
```

```
end
```

```
splitter = WordSplitter.new
splitter.string = "how do you do"
```

```
p splitter.find_all { |word| word.include?("d") }
```

← возвращает true.

```
p splitter.reject { |word| word.include?("d") }
```

← Отклоняет элементы, для которых блок возвращает true.

```
p splitter.map { |word| word.reverse }
```

← Возвращает массив со всеми возвращаемыми значениями блока.

```
["do", "do"]
["how", "you"]
["woh", "od", "uoy", "od"]
```

Этим методом блок не нужен.

```
{ p splitter.any? { |word| word.include?("e") }
  p splitter.count
  p splitter.first
  p splitter.sort }
```

← Подсчет всех элементов.
← Первый элемент.
← Отсортированный массив элементов.

Метод возвращает true, если блок возвращает true хотя бы для одного элемента.

```
false
4
"how"
["do", "do", "how", "you"]
```

Модули

```
module Enumerable
  def find_all
    matching_items = []
    self.each do |item|
      if yield(item)
        matching_items << item
      end
    end
    matching_items
  end

  def reject
    kept_items = []
    self.each do |item|
      unless yield(item)
        kept_items << item
      end
    end
    kept_items
  end

  def map
    results = []
    self.each do |item|
      results << yield(item)
    end
    results
  end

  ...

end
```

Создаем новый массив для хранения элементов, для которых блок возвращает true.

Обрабатываем каждый элемент.

Элемент передается блоку. Если результат равен true...

...он добавляется в массив.

Создаем новый массив для хранения элементов, для которых блок возвращает false.

Обрабатываем каждый элемент.

Элемент передается блоку. Если результат равен false...

...он добавляется в массив.

Создаем новый массив для хранения возвращаемых значений блока.

Обрабатываем все элементы.

Элемент передается блоку, а возвращаемое значение добавляется в новый массив.

Возвращаем массив возвращаемых значений блока.

И еще очень много методов!

Модули

Kernel

The Kernel module is included by class Object, so its methods are available in every Ruby object.

The Kernel instance methods are documented in class Object while the module methods are documented here. These methods are called without a receiver and thus can be called in functional form:

```
sprintf "%.1f", 1.234 #=> "1.2"
```

fronzen-string-literal: true

Public Instance Methods

Модули

Kernel

The Kernel module is included by class Object, so its methods are available in every Ruby object.

The Kernel instance methods are documented in class Object while the module methods are documented here. These methods are called without a receiver and thus can be called in functional form:

```
sprintf "%.1f", 1.234 #=> "1.2"
```

fronzen-string-literal: true

Public Instance Methods

Исключения

Рассмотрим пример

```
class SmallOven

  attr_accessor :contents

  def turn_on
    puts "Turning oven on."
    @state = "on"
  end
  def turn_off
    puts "Turning oven off."
    @state = "off"
  end

  def bake
    Если печь не была включена, { unless @state == "on"
    предупредить пользователя. { return "You need to turn the oven on first!"
                                } end
    Если духовка пуста, предупредить { if @contents == nil
    пользователя. { return "There's nothing in the oven!"
                    } end
    "golden-brown #{contents}"
  end
end
```

Но при запуске могут возникнуть проблемы

Забыли поставить блюдо в духовку!

```
dinner = ['turkey', nil, 'pie']  
oven = SmallOven.new  
oven.turn_on  
dinner.each do |item|  
  oven.contents = item  
  puts "Serving #{oven.bake}."  
end
```

*Выглядит не-
аппетитно...*

```
Turning oven on.  
Serving golden-brown turkey.  
Serving There's nothing in the oven!.  
Serving golden-brown pie.
```

Проблемы могут возникать разные

```
dinner = ['turkey', 'casserole', 'pie']
oven = SmallOven.new
oven.turn_off ← Случайно выключили духовку («off») вместо того, чтобы включить («on»)!
dinner.each do |item|
  oven.contents = item
  puts "Serving #{oven.bake}."
end
```

Да это вообще
несъедобно! {

```
Turning oven off.
Serving You need to turn the oven on first!.
Serving You need to turn the oven on first!.
Serving You need to turn the oven on first!.
```


Как с этим бороться?

Основной проблемой предыдущего примера является то, что выполнение программы не останавливается в случае, если что-то пойдет не так. Для реальных задач такой подход может оказаться фатальным. Но мы помним, что Ruby это по-настоящему прекрасный и добрый к разработчику язык. Что он может предложить нам для обработки исключительных ситуаций?

Например, метод raise

Он прерывает выполнение программы с сообщением об ошибке при обнаружении аномального состояния. Похоже, такой способ намного надежнее передачи сообщений об ошибках в возвращаемых значениях методов.

```
class Dog
  attr_reader :name, :age

  def name=(value)
    if value == ""
      raise "Name can't be blank!"
    end
    @name = value
  end
```

Если значение «value» недействительно...

...выполнение прерывается в этой точке.

Эта команда не будет выполнена, если был вызван метод «raise».

Использование «raise»

```
class SmallOven

  attr_accessor :contents

  def turn_on
    puts "Turning oven on."
    @state = "on"
  end
  def turn_off
    puts "Turning oven off."
    @state = "off"
  end

  def bake
    unless @state == "on"
      raise "You need to turn the oven on first!"
    end
    if @contents == nil
      raise "There's nothing in the oven!"
    end
    "golden-brown #{contents}"
  end
end
```

Если пользователь пытается приготовить блюдо при выключенной духовке, происходит ошибка.

Если пользователь пытается готовить с пустой духовкой, происходит ошибка.

Эта строка не выполняется, если ранее произошла ошибка.

Теперь даже появились RuntimeError

```
oven = SmallOven.new
oven.turn_off ← Случайно выключили духовку.
oven.contents = 'turkey'
puts "Serving #{oven.bake}."
```

```
Turning oven off.
oven.rb:16:in `bake': You need to turn the oven on first! (RuntimeError)
      from oven.rb:29:in `<main>'
```

```
oven = SmallOven.new
oven.turn_on
oven.contents = nil ← Духовка пуста.
puts "Serving #{oven.bake}."
```

```
Turning oven on.
oven.rb:19:in `bake': There's nothing in the oven! (RuntimeError)
      from oven.rb:29:in `<main>'
```

Но использование «raise» может создавать новые проблемы

Тот же самый код...

```
dinner = ['turkey', nil, 'pie']
oven = SmallOven.new
oven.turn_on
dinner.each do |item|
  oven.contents = item
  puts "Serving #{oven.bake}."
end
```

Выполнение
останавливается
на «nil», без об-
работки «pie»!

```
Turning oven on.
Serving golden-brown turkey.
oven.rb:19:in `bake': There's nothing in the oven! (RuntimeError)
    from oven.rb:31:in `block in <main>'
    from oven.rb:29:in `each'
    from oven.rb:29:in `<main>'
```

При этом старая версия продолжала работу

Забыли поставить блюдо в духовку!

```
dinner = ['turkey', nil, 'pie']  
oven = SmallOven.new  
oven.turn_on  
dinner.each do |item|  
  oven.contents = item  
  puts "Serving #{oven.bake}."  
end
```

Выглядит неаппетитно...

```
Turning oven on.  
Serving golden-brown turkey.  
Serving There's nothing in the oven!.  
Serving golden-brown pie.
```

Теорема Эскобара

Обе версии имеют свои преимущества и недостатки, работают корректно в одних ситуациях и не могут справиться с другими. Причем ни один метод не показывает какого-либо решающего преимущества перед другим. Неужели придется выбрать один и мириться со всеми потенциальными проблемами?

Если сам Эскобар не мог формальными методами разрешить свою знаменитую дилемму, то может быть, Ruby знает решение?

Исключения

Если просто вызвать `raise` в сценарии, результат будет выглядеть примерно так:

```
raise "oops!"
```

```
myscript.rb:1:in `<main>': oops! (RuntimeError)
```

Метод `raise` в действительности создает объект **исключения** — то есть объект, представляющий ошибку. Если исключение не будет обработано, то программа аварийно завершается.

Исключение — объект,
представляющий ошибку.

Исключения (продолжение)

При выходе из метода Ruby также ищет условие `rescue` там, откуда был вызван метод. Таким образом, если вы вызываете метод, который (как вы подозреваете) может инициировать исключение, заключите вызов в блок `begin/end` и добавьте условие `rescue`.

```
def risky_method
  raise "oops!"
  puts "I'll be skipped."
end

begin
  risky_method
rescue
  puts "Rescued an exception!"
end
```

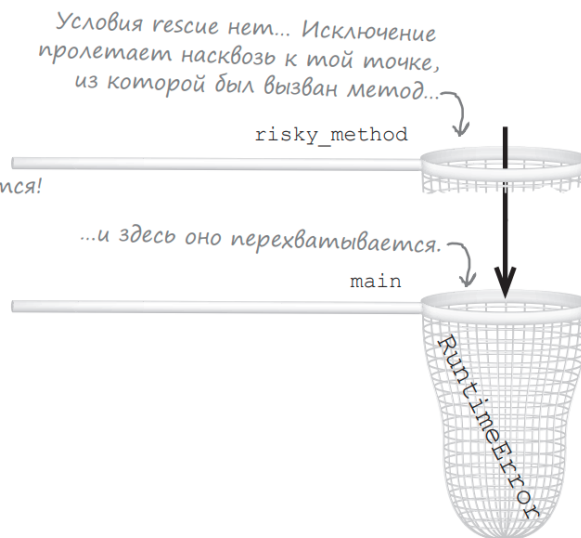
Выполнение метода не-медленно прерывается.

Никогда не выполняется.

Иницирует исключение.

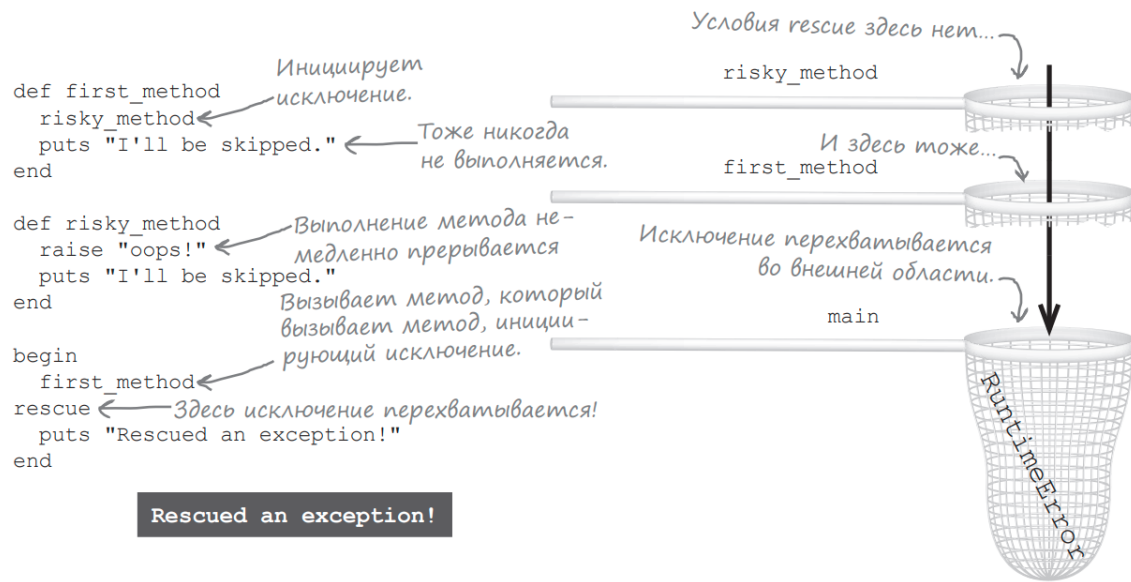
А здесь исключение перехватывается!

Rescued an exception!



Исключения (продолжение)

Исключение может передаваться по цепочке из нескольких методов. Если в области вызова метода не найдется подходящего условия `rescue`, то Ruby немедленно выходит из *этого* метода и ищет условие `rescue` в области *его* вызова. Это продолжается по цепочке до тех пор, пока не будет найдено подходящее условие `rescue`. (Если условие *не* будет найдено, происходит аварийное завершение работы программы.)



Вернемся к примеру

```
class SmallOven
  ...
  def bake
    unless @state == "on"
      raise "You need to turn the oven on first!"
    end
    if @contents == nil
      raise "There's nothing in the oven!"
    end
    "golden-brown #{contents}"
  end
end

dinner = ['turkey', nil, 'pie']
oven = SmallOven.new
oven.turn_on
dinner.each do |item|
  oven.contents = item
  puts "Serving #{oven.bake}."
end
```

*Забыли поставить
блюдо в духовку!*



Сейчас результат такой

Все останавли-
вается на эле-
менте «nil» без
обработки «pie»!



```
Turning oven on.  
Serving golden-brown turkey.  
oven.rb:19:in `bake': There's nothing in the oven! (RuntimeError)  
    from oven.rb:31:in `block in <main>'  
    from oven.rb:29:in `each'  
    from oven.rb:29:in `<main>'
```

Внесем пару изменений

```
dinner = ['turkey', nil, 'pie']
oven = SmallOven.new
oven.turn_on
dinner.each do |item|
  begin
```

Код остался неизменным,
но теперь он заключен
в блок begin/end

```
    { oven.contents = item
      puts "Serving #{oven.bake}."
    rescue
```

← Перехватывает любые исключения,
возникающие в двух предыдущих строках.

```
      puts "Error: There's nothing in the oven!"
```

← Выводим сообщение об ошибке.

```
    end
```

```
  end
```

Первое блюдо готово.

Для блюда «nil» выводится сообщение об ошибке.

Переходим к приготовлению третьего блюда.

```
Turning oven on.
Serving golden-brown turkey.
Error: There's nothing in the oven!
Serving golden-brown pie.
```

Передача сообщений об ошибке

Если в методе `bake` обнаруживается ошибка, строка с описанием проблемы передается методу `raise`:

```
class SmallOven
```

```
  ...
```

```
  def bake
```

```
    unless @state == "on"
```

```
      raise "You need to turn the oven on first!"
```

```
    end
```

```
    if @contents == nil
```

```
      raise "There's nothing in the oven!"
```

```
    end
```

```
    "golden-brown #{contents}"
```

```
  end
```

```
end
```

Если духовка выключена, это сообщение передается «`raise`»...

Если духовка пуста, то передается другое сообщение...

Передача сообщений об ошибке (продолжение)

Пока в программе эти сообщения не используются; вместо этого в условии `rescue` всегда выводится одна строка, которая сообщает, что духовка пуста.

Но если духовка на самом деле *выключена*, а не пуста, то будет выведено неправильное сообщение об ошибке!

```
dinner = ['turkey', 'casserole', 'pie']
oven = SmallOven.new
oven.turn_off ← Если духовка выключена...
dinner.each do |item|
  begin
    oven.contents = item
    puts "Serving #{oven.bake}."
  rescue
    puts "Error: There's nothing in the oven!" ←
  end
end
```

...программа игнорирует сообщения, переданные «raise», и использует одно сообщение во всех случаях!

Неправильно! Проблема в том, что духовка выключена!

```
Turning oven off.
Error: There's nothing in the oven!
Error: There's nothing in the oven!
Error: There's nothing in the oven!
```

Передача сообщений об ошибке (продолжение)

Если при вызове `raise` передается строка, то метод использует эту строку для назначения атрибута `message` созданного им исключения:

```
unless @state == "on"
  raise "You need to turn the oven on first!"
end
if @contents == nil
  raise "There's nothing in the oven!"
end
```

*Создает объект исключения
и задает его атрибут «message».*

То же самое.

Передача сообщений об ошибке (продолжение)

Нужно сделать совсем немного: удалить фиксированное сообщение об ошибке и вывести вместо него текущее содержимое атрибута `message` объекта исключения.

Чтобы сохранить исключение в переменной, нужно добавить в строку `rescue` символы `=>`, за которыми следует имя нужной переменной. (Обозначение `=>` используется в некоторых литералах хешей, но в данном контексте оно не имеет никакого отношения к хешам.) Если объект исключения доступен, мы можем вывести его атрибут `message`.

```
begin
  raise "oops!"
rescue => my_exception
  puts my_exception.message
end
```

Создает исключение со строкой «oops!» в атрибуте message.

Исключение сохраняется в переменной.

Выводим сообщение из исключения.

oops !

Передача сообщений об ошибке (продолжение)

```
dinner = ['turkey', 'casserole', 'pie']
oven = SmallOven.new
oven.turn_off ← Духовка выключена.
dinner.each do |item|
  begin
    oven.contents = item
    puts "Serving #{oven.bake}."
    rescue => error ← Исключение сохраняется в переменной.
    puts "Error: #{error.message}" ← Выводим сообщение,
    end                                     которое хранится
end                                     в исключении.
```

Проблема описана в сообщении из исключения.

```
Turning oven off.
Error: You need to turn the oven on first!
Error: You need to turn the oven on first!
Error: You need to turn the oven on first!
```

Разная логика rescue для разных исключений

```
class SmallOven
```

```
  ...
```

```
  def bake
```

```
    unless @state == "on"
```

```
      raise "You need to turn the oven on first!"
```

```
    end
```

```
    if @contents == nil
```

```
      raise "There's nothing in the oven!"
```

```
    end
```

```
    "golden-brown #{contents}"
```

```
  end
```

```
end
```

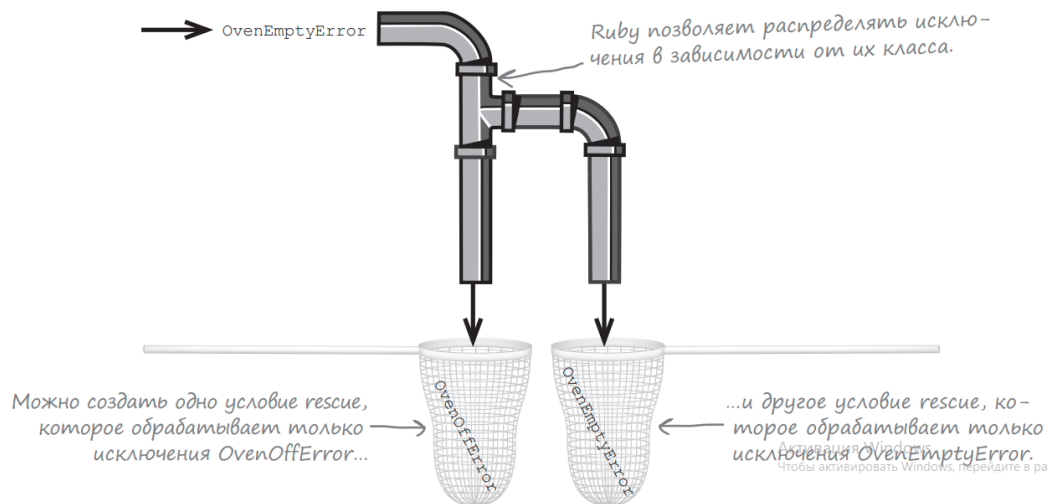
Включить духовку и попробовать снова? Вполне разумный план для обработки этого исключения...

...но только не для этого!

Необходимо как-то различать исключения, которые может выдавать метод `bake`, чтобы по-разному обрабатывать их. А для *этого* можно использовать класс исключения...

Разная логика rescue для разных исключений (продолжение)

Ранее мы уже упоминали о том, что исключения представляют собой объекты. Но ведь каждый объект — экземпляр некоторого класса, верно? Вы можете указать, какие классы исключений обрабатываются каждым конкретным условием `rescue`. В этом случае условие `rescue` будет игнорировать любые исключения, не являющиеся экземпляром указанного класса (или одного из его subclasses). Эта возможность позволяет направить исключение условию `rescue`, которое сможет обработать его так, как вам нужно.



Классы исключений

Когда мы вызываем метод `raise`, он создает объект исключения... и если это исключение не будет перехвачено, вы увидите класс объекта исключения при завершении программы.

```
raise "oops!"
```

```
myscript.rb:1:in `': oops! (RuntimeError)
```

*Объект исключения
является экземпляром
класса `RuntimeError`.*



Классы исключений (продолжение)

По умолчанию `raise` создает экземпляр класса `RuntimeError`. Впрочем, при желании можно выбрать другой класс, который будет использован методом `raise`. Просто передайте имя класса в первом аргументе, перед строкой, которая определяет сообщение данного исключения.

Укажите
имя класса. ↓
`raise ArgumentError, "This method takes a String!"`

```
myscript.rb:1:in `<main>':  
This method takes a String! (ArgumentError)
```

Укажите класс. ↓

Иницируется
исключение задан-
ного класса! ↑

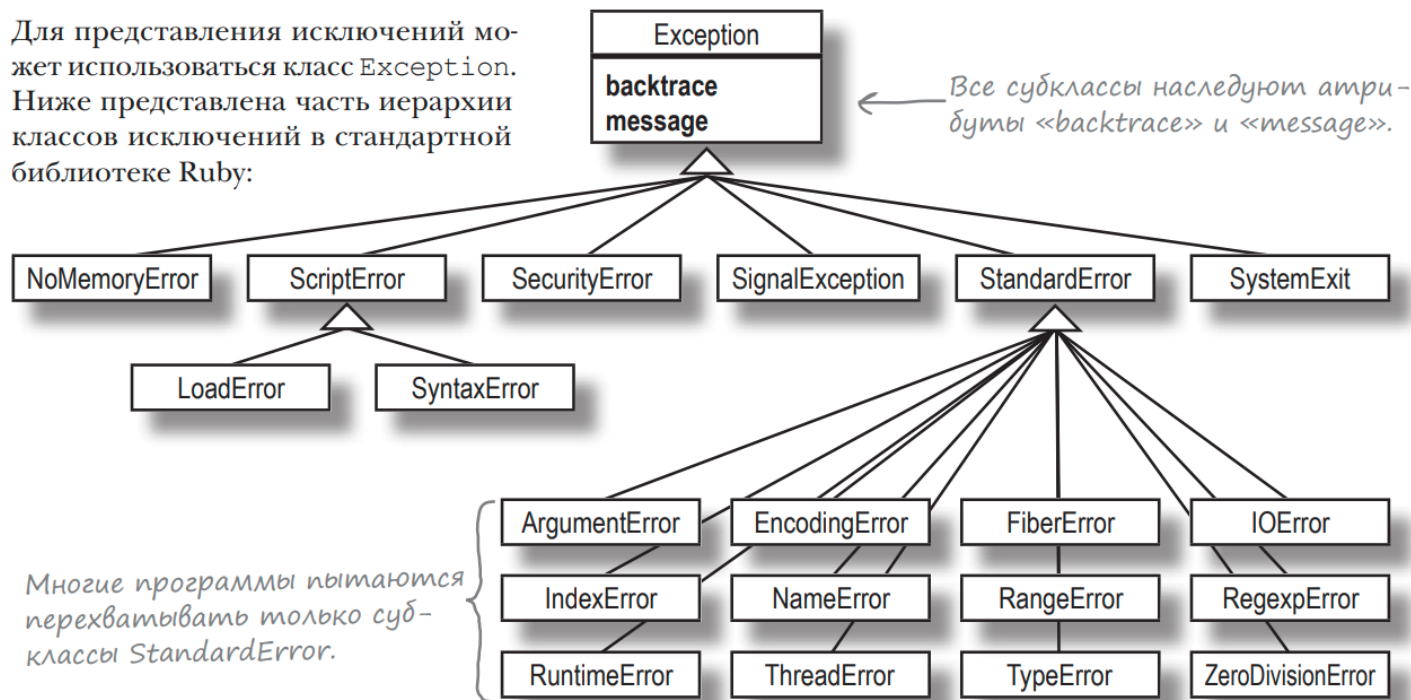
`raise ZeroDivisionError, "Can't cut a pie into 0 portions!"`

```
myscript.rb:1:in `<main>':  
Can't cut a pie into 0 portions! (ZeroDivisionError)
```

Происходит исключение
заданного класса! ↑

Классы исключений (продолжение)

Для представления исключений может использоваться класс Exception. Ниже представлена часть иерархии классов исключений в стандартной библиотеке Ruby:



Классы исключений (продолжение)

Итак, если вы сделаете свой класс исключения субклассом `Exception`, он будет работать с `raise`...

```
class MyError < Exception ← Должен быть суб-  
end                       классом Exception.
```

```
raise MyError, "oops!"
```

myscript.rb:4:in ``<main>`': oops! (MyError)

...но учтите, что в Ruby классы исключений обычно являются субклассами `StandardError`, а не прямыми субклассами `Exception`. По общепринятым соглашениям `StandardError` представляет категорию ошибок, которые могут быть обработаны типичной программой. Другие субклассы `Exception` представляют проблемы, неподконтрольные вашей программе, — например, нехватку памяти в системе или завершение ее работы.

Классы исключений (продолжение)

Итак, хотя вы *можете* использовать `Exception` как суперкласс для своих исключений, обычно вместо него следует использовать `StandardError`.

```
class MyError < StandardError
```

← Обычно стоит subclass'ить `StandardError`, а не `Exception`.

```
end
```

```
raise MyError, "oops!"
```

```
myscript.rb:4:in `<main>': oops! (MyError)
```

Работа с собственными классами исключений

Теперь, когда вы научились создавать собственные классы исключений, можно заняться перехватом *нужных* классов. Указывая имя класса за ключевым словом `rescue` в условии `rescue`, вы сообщаете, что условие должно перехватывать только исключения, являющиеся экземплярами этого класса (или одного из его субклассов).

В этом коде инициализированное исключение (`PorridgeError`) не совпадает с типом, указанным в условии `rescue` (`BeddingError`), поэтому исключение не перехватывается:

```
class PorridgeError < StandardError
end
class BeddingError < StandardError
end

def eat
  raise PorridgeError, "too hot"
end
def sleep
  raise BeddingError, "too soft"
end

begin
  eat
  rescue BeddingError => error
    puts "This bed is #{error.message}!"
  end
end
```

Иницилируем исключение `PorridgeError`.

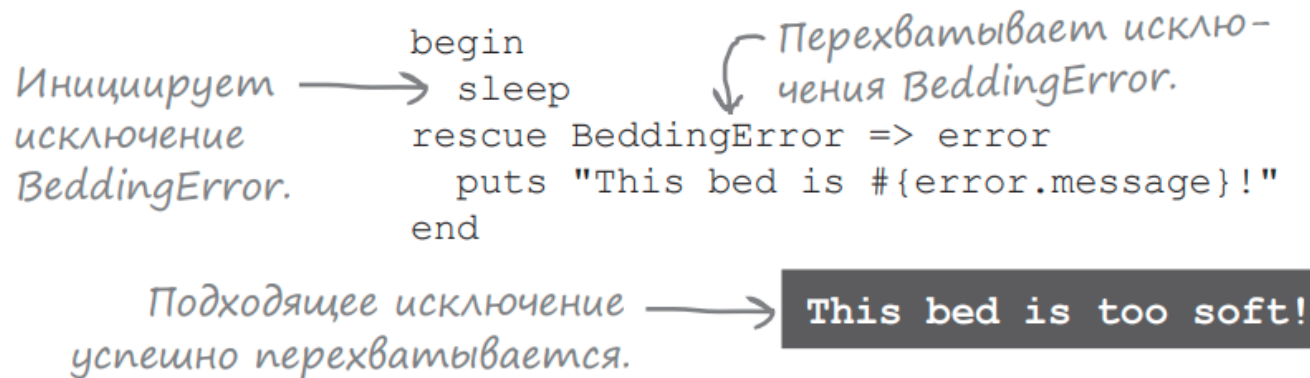
Перехватывает только исключения `BeddingError`.

Исключение `PorridgeError` остается необработанным.

```
goldilocks.rb:7:in `eat': too hot (PorridgeError)
from goldilocks.rb:14:in `<main>'
Активация Windows
```

Работа с собственными классами исключений (продолжение)

...но все исключения, которые *соответствуют* классу, указанному в условии `rescue`, будут обработаны:



Старайтесь всегда указывать тип исключения в условиях `rescue`. В этом случае ваша программа будет перехватывать только те исключения, которые она умеет обрабатывать.

Несколько условий rescue в одном блоке begin/end

Этот код перехватывает все экземпляры `BeddingError`, но игнорирует исключения `PorridgeError`. А мы хотим, чтобы перехватывались *оба* типа исключений...

```
class PorridgeError < StandardError
end
class BeddingError < StandardError
end
```

```
def eat
  raise PorridgeError, "too hot"
end
def sleep
  raise BeddingError, "too soft"
end
```

Иницирует исключение `PorridgeError`.

```
begin
  eat
rescue BeddingError => error
  puts "This bed is #{error.message}!"
end
```

Перехватывает только
исключения `BeddingError`.

Исключение `PorridgeError`
остается необработанным.

```
goldilocks.rb:7:in `eat': too hot (PorridgeError)
from goldilocks.rb:14:in `<main>'
```

Несколько условий rescue в одном блоке begin/end (продолжение)

В один блок begin/end можно добавить несколько условий rescue, в каждом из которых указывается отдельный тип исключения.

```
begin
  eat
  rescue BeddingError => error ← Сюда направляются исключения BeddingError.
    puts "This bed is #{error.message}!"
  rescue PorridgeError => error ← Сюда направляются исключения PorridgeError.
    puts "This porridge is #{error.message}!"
end
```

This porridge is too hot!

Это позволяет выполнять разный код в зависимости от того, к какому типу относится перехваченное исключение.

```
begin
  sleep ← На этот раз иницируется исключение BeddingError.
  rescue BeddingError => error ← Оно обрабатывается здесь.
    puts "This bed is #{error.message}!"
  rescue PorridgeError => error
    puts "This porridge is #{error.message}!"
end
```

This bed is too soft!

Переход на пользовательские классы исключений

Определяем два
новых класса
исключений.

```
{ class OvenOffError < StandardError
  end
  class OvenEmptyError < StandardError
  end
```

Иницилируем
исключение
одного типа,
если духовка
выключена...

...и исключение
другого типа,
если духовка
пуста.

```
class SmallOven
  ...
  def bake
    unless @state == "on"
      → raise OvenOffError, "You need to turn the oven on first!"
    end
    if @contents == nil
      → raise OvenEmptyError, "There's nothing in the oven!"
    end
    "golden-brown #{contents}"
  end
end
```

Переход на пользовательские классы исключений (продолжение)

Снова забыли поставить блюдо в духовку!

```
dinner = ['turkey', nil, 'pie']
oven = SmallOven.new
oven.turn_off
dinner.each do |item|
  begin
    oven.contents = item
    puts "Serving #{oven.bake}."
  rescue OvenEmptyError => error
    puts "Error: #{error.message}"
  rescue OvenOffError => error
    oven.turn_on
  end
end
```

А духовка была выключена!

Выводит сообщение из исключения, как и в предыдущей версии кода.

Раз духовка выключена, мы включаем ее.

Условие rescue для OvenOffError включает духовку.

Условие rescue для OvenEmptyError выводит предупреждение.

```
Turning oven off.
Turning oven on.
Error: There's nothing in the oven!
Serving golden-brown pie.
```

Перезапуск после исключения

Впрочем, мы кое-что упустили из вида... Наше условие `rescue` для `OvenOffError` снова включило духовку, и *остальные* блюда были приготовлены успешно. Но так как исключение `OvenOffError` произошло в то время, когда мы пытались приготовить блюдо, эта часть обеда была в итоге пропущена! Нужно вернуться и снова попытаться приготовить индейку (turkey) после того, как духовка была включена.

Условие `rescue` для `OvenOffError` включает духовку. Но тогда одно блюдо оказывается пропущенным!



```
Turning oven off.  
Turning oven on.  
There's nothing in the oven!  
Serving golden-brown pie.
```


Перезапуск после исключения (продолжение)

Ключевое слово `retry` делает именно то, что нужно. Когда вы включаете `retry` в условие `rescue`, выполнение возвращается к началу блока `begin/end`, и находящиеся там команды выполняются снова. Например, если исключение возникло из-за попытки деления на 0, можно изменить делитель и попытаться выполнить операцию снова:

```
amount_won = 100
portions = 0
begin
  portion_size = amount_won / portions
  puts "You get $#{portion_size}."
rescue ZeroDivisionError
  puts "Revising portion count from 0 to 1."
  portions = 1
retry
end
```

Вызовет исключение `ZeroDivisionError`.

Исправляем проблему, породившую исключение.

После исправления делителя управления возвращается к началу блока «begin».

```
Revising portion count from 0 to 1.
You get $100.
```

Перезапуск после исключения (продолжение)

Будьте внимательны при использовании `retry`. Если вы не исправите проблему, породившую исключение (или если в коде `rescue` допущена ошибка), то исключение будет выдано снова, управление снова передастся `retry`... и программа войдет в бесконечный цикл! В таком случае нажмите клавиши `Ctrl-C`, чтобы прервать работу Ruby.

Если добавить в приведенный выше код `retry` без исправления проблемы с делителем, произойдет заикливание:

```
amount_won = 100
portions = 0
begin
  portion_size = amount_won / portions
  puts "You get $#{portion_size}."
  rescue ZeroDivisionError
    puts "Revising portion count from 0 to 1."
    retry
  end
```

Если пропустить выражение «portions = 1»... → ...перезапуск произойдет, а проблема сохранится!

Бесконечный цикл!

Нажмите `Ctrl-C`, чтобы прервать работу программы.

```
...
Revising portion count from 0 to 1.
Revising portion count from 0 to 1.
Revising portion count from 0 to 1.
^Cwin.rb:4:in `new': Interrupt
```

Использование retry

```
dinner = ['turkey', nil, 'pie']
oven = SmallOven.new
oven.turn_off
dinner.each do |item|
  begin
    oven.contents = item
    puts "Serving #{oven.bake}."
    rescue OvenEmptyError => error
      puts "Error: #{error.message}"
    rescue OvenOffError => error
      oven.turn_on
      retry ← Перезапускаем блок
    end
  end
end
```

«begin» после включения духовки.

*Условие rescue для OvenOffError
включает духовку.*

*Блок «begin» перезапускается, а индейка
успешно готовится.*

```
Turning oven off.
Turning oven on.
Serving golden-brown turkey.
Error: There's nothing in the oven!
Serving golden-brown pie.
```

Условие ensure

Условие `ensure` будет выполнено, если в программе происходит исключение:

```
begin
  raise "oops!"
rescue
  puts "rescued an exception"
ensure
  puts "I run regardless"
end
```

Выполняется
условие `rescue`...

**rescued an exception
I run regardless**

...а за ним выполняется условие `ensure`.

Условие ensure (продолжение)

Условие `ensure` выполняется и в том случае, если исключение *не* происходит:

```
begin
  puts "everything's fine"
rescue
  puts "rescued an exception"
ensure
  puts "I run regardless"
end
```

Выполняется тело
блока `<begin>...`



everything's fine
I run regardless

...а за ним выполняется условие `ensure`.

Условие ensure (продолжение)

Даже если исключение не перехватывается, условие ensure все равно будет выполнено до прерывания работы Ruby!

```
begin
  raise "oops!"
ensure
  puts "I run regardless"
end
```

*Условие ensure
выполняется...*



```
I run regardless
script.rb:2:in `<main>': oops! (RuntimeError)
```

...и только после этого работа Ruby завершается.

Попробуем на нашем примере

```
begin
  oven.turn_on
  oven.contents = 'turkey'
  puts "Serving #{oven.bake}."
rescue OvenEmptyError => error
  puts "Error: #{error.message}"
ensure
  oven.turn_off
end
```

Достаточно оставить эту
строку в условии ensure.

Блок «begin» завершается...
...за ним следует условие ensure.

```
Turning oven on.
Serving golden-brown turkey.
Turning oven off.
```

Попробуем на нашем примере (продолжение)

```
begin
  oven.turn_on
  oven.contents = nil
  puts "Serving #{oven.bake}."
rescue OvenEmptyError => error
  puts "Error: #{error.message}"
ensure
  oven.turn_off
end
```

Происходит исключение.

Достаточно оставить эту строку в условии ensure.

Выполняется условие rescue...

...а за ним следует условие ensure.

```
Turning oven on.
Error: There's nothing in the oven!
Turning oven off.
```